Learning Python Programming with MNIST

Applied Deep Learning — Chapter 3

Yin Yang

Foundation Books Project

Roadmap

Why MNIST

Python, Tensors, and Shapes

Dataset and Splits

From Pixels to Probabilities

The MLP Baseline

Training Loop

Keras 3, One Model, Three Backends

Direct PyTorch

Diagnosis and Limits

Summary

Why MNIST

MNIST: The "Hello World" of Deep Learning

- Small enough to run quickly.
- Visual enough to inspect by eye.
- Rich enough to require the full supervised routine: load, train, evaluate, inspect mistakes.
- Whole experiment fits in one sitting.

Not a difficult modern benchmark. A small lab for reading real training code: every shape and equation should connect to one digit image.

The Running Example

- One grayscale image of the digit 7.
- 28 rows $\times$ 28 columns $= 784$ pixel values.
- Label: integer 7.
- Class set: $\{0, 1, 2, \ldots, 9\}$.

Definition: supervised classification
Examples come with labels drawn from a fixed finite set of classes. The model learns a function from input examples to predicted classes.

Accuracy is Easy; Reporting is the Habit

Definition: accuracy
Fraction of test examples whose predicted class equals the true class. $9{,}730/10{,}000 = 97.3\%$.

Accuracy is easy to read but not the only evidence. A run report should include:

- split sizes, tensor shapes, normalization
- model, optimizer, learning rate, batch size, epochs
- runtime, seed, hardware
- a few mistakes the model still makes

Python, Tensors, and Shapes

Arrays, Tensors, Shapes

Definition
A *tensor* is a multidimensional numerical array. Its *shape* is the tuple of dimension sizes.

Standard Keras MNIST loader returns:

- `x_train.shape == (60000, 28, 28)`
- `y_train.shape == (60000,)`

Convention: the first dimension counts examples. Remaining dimensions describe one example.

Inspect Before You Train

Habit: print shapes, dtypes, and label samples *before* the first epoch.

- Pixel range correct? `uint8` $\in [0, 255]$ before normalization, `float32` $\in [0, 1]$ after.
- Label shape correct? Sparse integers, not one-hot, for `SparseCategoricalCrossentropy`.
- Number of images and labels match?

Most beginner failures are not exotic architecture problems — they are shape, label, or normalization problems caught here.

Batches

A *batch* groups examples for one update.

Shape-bug check: is the first dimension the batch, and do the remaining dimensions match the image or feature shape you expected?

Dataset and Splits

Train, Validation, Test

Training set used to update parameters.

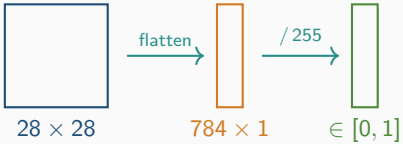
Validation set used during model selection.

Test set reserved for the final report.

Chapter split (from the official 60,000 training images):

- 50,000 train, 10,000 validation
- 10,000 official test — kept locked until the end.

From Pixels to Vectors



Flattening rearranges the values, not their meaning. Normalization keeps inputs in a scale matching common initializers and optimizers:

$$0/255 = 0, \quad 128/255 \approx 0.502, \quad 255/255 = 1.$$

Variants and Domain Shift

- **Fashion-MNIST**: same 28×28 format, clothing classes.
- **EMNIST**: NIST-style letters and digits.
- **QMNIST**: reconstructed extra NIST digits.
- **Kuzushiji-MNIST**: cursive Japanese characters.
- **Arabic handwritten digits**: writer and writing-system shift.

Domain shift
Training data and deployment data differ in important ways. Good MNIST accuracy does not transfer automatically.

Practical question: what changed about the data?

From Pixels to Probabilities

The Math of the MLP

Two hidden layers with ReLU plus a logits head:

$$
\begin{aligned}
h_1 &= \mathrm{ReLU}(W_1 x + b_1) \\
h_2 &= \mathrm{ReLU}(W_2 h_1 + b_2) \\
z &= W_3 h_2 + b_3
\end{aligned}
$$

Trainable parameters: W_1, W_2, W_3 and b_1, b_2, b_3.

$z \in \mathbb{R}^{10}$ is the vector of logits. Logits are not probabilities — they can be negative and need not sum to one.

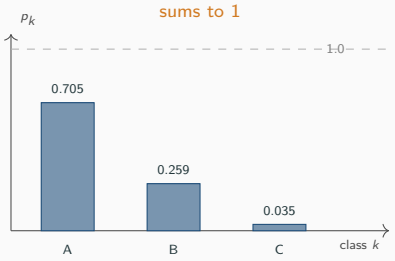
Softmax and Cross-Entropy

Softmax

$$p_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$$

Logits $(2.0, 1.0, -1.0)$, exps $(7.39, 2.72, 0.37)$, sum ≈ 10.48.
Probs $\approx (0.705, 0.259, 0.035)$.

Sparse cross-entropy: $\ell = -\log(p_y)$.
$p_y = 0.90 \Rightarrow \ell \approx 0.105$; $p_y = 0.10 \Rightarrow \ell \approx 2.303$.



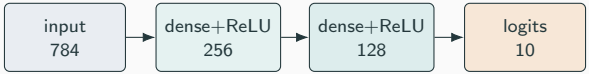
Logits Stay Logits in the Code

- Final layer returns logits.
- Loss receives `from_logits=True` (Keras) or uses `nn.CrossEntropyLoss` (PyTorch).
- Frameworks combine softmax and log internally for numerical stability.
- Apply softmax *after* prediction when probabilities are needed for display.

Common bug: adding a softmax layer *and* using `from_logits=True`. The loss then double-applies softmax.

The MLP Baseline

Architecture: $784 \to 256 \to 128 \to 10$



Parameter count:

- $784 \times 256 + 256 = 200{,}960$
- $256 \times 128 + 128 = 32{,}896$
- $128 \times 10 + 10 = 1{,}290$
- **Total:** $235{,}146$ **trainable parameters**.

Why This Architecture?

- Simple enough that every parameter and shape can be checked by hand.
- Ignores spatial structure — a CNN would do better. That comes next chapter.
- ReLU because it is fast, simple, and effective.
- Adam at lr 10^{-3}: trains this network reliably without schedule tuning.

Training Loop

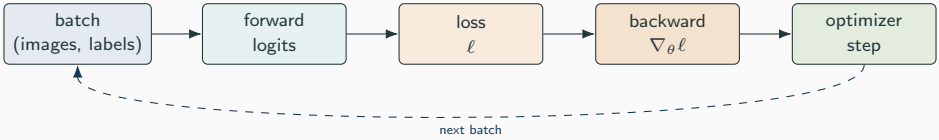
The Update Step

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \cdot \nabla_\theta \mathcal{L}_B(\theta_{\text{old}}).$$

- θ: all weights and biases.
- η: learning rate.
- $\nabla_\theta \mathcal{L}_B$: gradient of the batch loss.

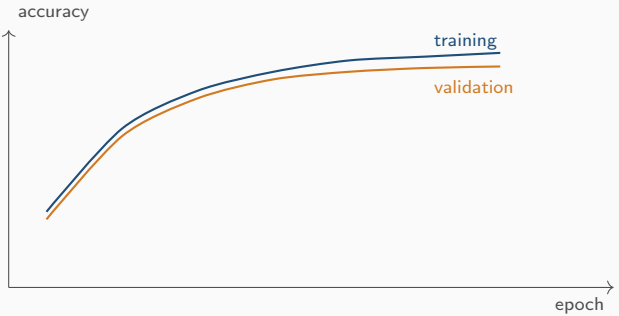
Gradient points toward higher loss; the optimizer subtracts a scaled version. One epoch on 50,000 examples with batch 128 is about 391 such updates.

Forward, Backward, Step



Keras hides this inside `model.fit`; PyTorch writes it explicitly with `optimizer.zero_grad`, `loss.backward`, and `optimizer.step`.

Reading the Curves



- Initial loss near $\log(10) \approx 2.30$ and accuracy near 10% is the expected starting point.
- Far outside that range $\Rightarrow$ wiring clue, not model quality.
- Validation accuracy is for model selection. Test accuracy is for the final report — use *once*.

Underfitting and Overfitting

Definitions
Underfitting: model fails to capture training-set structure. *Overfitting:* model fits training-specific details that do not generalize. *Generalization:* performance on new examples.

For this MLP, underfitting (low train and val accuracy) is more likely than overfitting. Look first at:

- normalization
- learning rate
- label encoding and loss configuration

Keras 3, One Model, Three Backends

Keras 3 Backend Selection

Keras 3 runs on TensorFlow, PyTorch, or JAX — the central model code is backend-neutral.

Set the backend before importing Keras. From the shell:

```
KERAS_BACKEND=tensorflow python mnist_keras3.py
KERAS_BACKEND=torch      python mnist_keras3.py
KERAS_BACKEND=jax        python mnist_keras3.py
```

Restart kernel after changing it.

Keras Walkthrough Cells

Six cells in the chapter notebook:

1. Select backend and import libraries.
2. Set constants (`SEED`, `BATCH_SIZE`, `EPOCHS`); print versions.
3. Load MNIST; normalize; carve out validation.
4. Define the `Sequential` model: `Flatten`, two `Dense+ReLU`, final `Dense(10)`.
5. `model.compile`: Adam, sparse cross-entropy with `from_logits=True`, accuracy metric.
6. `fit`, `evaluate`, inspect predictions.

See `code/chapter_mnist_python/` for the runnable script.

Hosted Notebooks

- **Colab:** new Python notebook; backend in the first cell; restart runtime if you change major packages.
- **Kaggle:** notebook settings panel for CPU/accelerator; `/kaggle/input/` for attached datasets, `/kaggle/working/` for outputs.

Record device, package versions, and elapsed time. Hosted runtimes change — do not promise future sessions match.

Direct PyTorch

Why Also Write the Loop by Hand?

Keras hides the training loop. Direct PyTorch exposes it:

- Where the batch enters the model.
- Where loss is computed.
- Where the optimizer updates weights.

Same architecture: `nn.Flatten`, `nn.Linear(784,256)`, `nn.ReLU`, `nn.Linear(256,128)`, `nn.ReLU`, `nn.Linear(128,10)`.

Device Selection

Try in order:

1. CUDA if available.
2. Apple MPS if available.
3. CPU otherwise.

`torch.manual_seed(SEED)` for reproducibility. Move both model and batches to the chosen `device`. The device used belongs in the run log alongside Python and PyTorch versions.

The Inner Loop

1. Move `images, labels` to device.
2. `logits = model(images)`.
3. `loss = loss_fn(logits, labels)`.
4. `optimizer.zero_grad()` — clear stale gradients.
5. `loss.backward()` — backpropagation.
6. `optimizer.step()` — apply update.

Forget zero_grad and gradients accumulate. Forget step and the weights never change.

Eval vs. Train Mode

`model.train()` training mode (dropout, batchnorm updates).

`model.eval()` evaluation mode.

`@torch.no_grad()` disables gradient tracking.

No visible effect for this MLP — but the habit prevents bugs in larger models with dropout, batch normalization, or running statistics.

Diagnosis and Limits

When the Loss Will Not Move

Symptom	Likely cause	First action
Loss barely decreases	Missing normalization; lr too small	Print pixel range; verify optimizer step
Initial loss far from $\log 10$	Label encoding or loss config mismatch	Print one batch of labels; verify ten-logit head
Train and val both low	Underfitting	Train longer; widen layers
Train high, val stalls	Overfit or test-set tuning	Stop earlier; simplify; recheck split discipline
Slow on hosted runtime	CPU session, package mismatch	Record device + versions
Strange mistakes	Data or preprocessing issue	Inspect misclassified images

Beginner-Safe Knobs

Things to vary, one at a time:

- Number of epochs.
- Batch size: 64, 128, 256.
- Hidden widths: $128, 64$ vs. $256, 128$.
- Optimizer learning rate around 10^{-3}.

Save weight decay, dropout, and learning-rate schedules for the training chapter. Don't change five things at once — a controlled experiment makes evidence explainable.

What MNIST Does Not Teach

- No data collection, messy labels, or class imbalance.
- No privacy, fairness, or robustness pressure.
- No deployment story.
- No need yet for convolutional inductive bias.

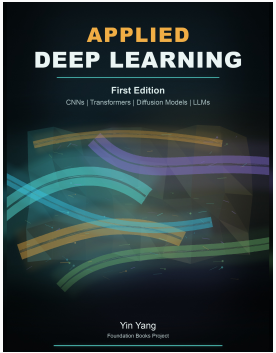
MNIST can flatter a broken workflow — many mistakes still leave enough signal for a decent-looking accuracy number. Use Fashion- MNIST, EMNIST, Arabic digits, etc., to test whether the same habits transfer.

Summary

Summary

- One 28×28 image $\rightarrow$ tensor $\rightarrow$ 784-vector $\rightarrow$ MLP $\rightarrow$ ten logits.
- Softmax converts logits to probabilities for display. Cross-entropy is the training loss.
- Keras 3 expresses the model concisely; backend chosen before import. Direct PyTorch makes the loop explicit.
- Inspect shapes, normalize, fix the seed, reserve the test set.
- Record versions, backend, hardware, batch size, epochs, loss, accuracy, elapsed time — and a few mistakes.
- MNIST is a wiring check. Variants test whether the habits transfer.

Next: keep the same workflow but exploit image structure — CNN basics on MNIST and CIFAR-10.



Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	`foundation-books.github.io/applied-deep-learning`
Code	`github.com/foundation-books/applied-deep-learning-code`
Print	`amazon.com/dp/B0GZF7QRSH`
Kindle	`amazon.com/dp/B0GZF9221X`
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.